

VCDSTIL User Manual

June 6, 2011

Verigy Germany GmbH.
Herrenberger Strasse 130
71034 Boeblingen.

Contents

1	Introduction	3
1.1	Audience	3
1.2	Concept	3
2	Installing VCDSTIL	7
3	VCDSTIL Operational Principles	9
3.1	Cyclization Of Event Based Data	9
3.2	Understanding Event Based Data	10
3.2.1	VCD Files	10
3.2.2	EVCD Files	10
3.3	Understanding STIL	10
3.4	VCDSTIL Cycle Matching	12
3.4.1	Force User-Defined Timing	12
3.4.2	Condition Based Waveform Matching	12
3.4.3	Waveform Matching	12
3.4.4	How Waveforms Are Mapped To STIL	13
3.5	How STIL Pattern Data (WFC) Is Formed	13
4	Using VCDSTIL	17
4.1	Introduction	17
4.2	Setting Up Signals	17
4.2.1	Import Signals from Event Based Files	18
4.2.2	Updating Signal Properties	18
4.3	Settings Up Groups	21
4.4	Defining Signal Timing	22
4.4.1	VCDSTIL Waveform Definition	23
4.4.2	Extracting Timing From Event Based files	25
4.4.3	VCDSTIL Forced Waveforms	26
4.4.4	VCDSTIL Conditional Waveforms	27
4.4.5	Multiport Mode	29
4.5	Settings Up AC Specs	30
4.6	Setting Up the VCDSTIL Setup File	31
4.6.1	Creating VCDSTIL Setup Template Files	31
4.6.2	Setup File Parameters	32
4.7	Using Command Line Interface (CLI) of VCDSTIL	38
4.7.1	Command Line Options Description	38
5	VCDSTIL Tutorial	39
5.1	Introduction	39
5.1.1	VCDSTIL concept	39
5.1.2	Modes of operation	39

5.2	DUT Description - I8254	40
5.2.1	DUT overview	40
5.2.2	DUT pin definitions	40
5.2.3	DUT timing definitions	40
5.3	Using automated timing extraction	41
5.3.1	Creating setup file from template	41
5.3.2	Extracting pins information from VCD	41
5.3.3	Extracting timing from VCD	44
5.3.4	Generate STIL	46
5.4	Using user-defined waveforms	48
5.4.1	Why use user-defined waveforms	48
5.4.2	Modifying the extracted timing	48
5.5	Using conditional waveforms	50
5.5.1	Why use conditional waveforms	50
5.5.2	Modifying the timing	50
6	Advanced Tutorials	53
6.1	Using VCDSTIL with a spreadsheet (Excel/ Open office)	53
6.1.1	Run VCDSTIL with CSV (spreadsheet) file	53
6.1.2	Commands Explanation	57
6.2	How to use the Master file withste	58
6.3	Multi (E)VCD files with master file	64
6.4	SSP tutorial	66
6.5	Input/Output Timing Definition Tutorial	69
7	Advanced Topics	73
7.1	Controlling the Direction of Inout Signals	73
7.1.1	Controlling Direction Using I/O Control	73
7.1.2	Controlling Direction Using Statemap	74
7.2	Masking Output Signals	74
7.2.1	Masking Transitions	75
7.2.2	Masking Using Conditions	75
7.2.3	Masking Using State Map	76
7.3	Changing Signal Values	77
7.4	Using Multiple Clock Domains	78
7.5	Using Multiple Waveform Tables	79
7.6	Optimizing Strobe Position	80

Chapter 1

Introduction

1.1 Audience

The VCDSTIL User Manual is intended for test and design engineers of integrated circuits.

1.2 Concept

The need to convert test patterns from simulation to ATE format is a well known challenge for test engineers. Functional test patterns are created using Verilog/VHDL simulators. Such simulations typically create event driven data with no cycle definition. An example of event stream is show in Figure 1.1.

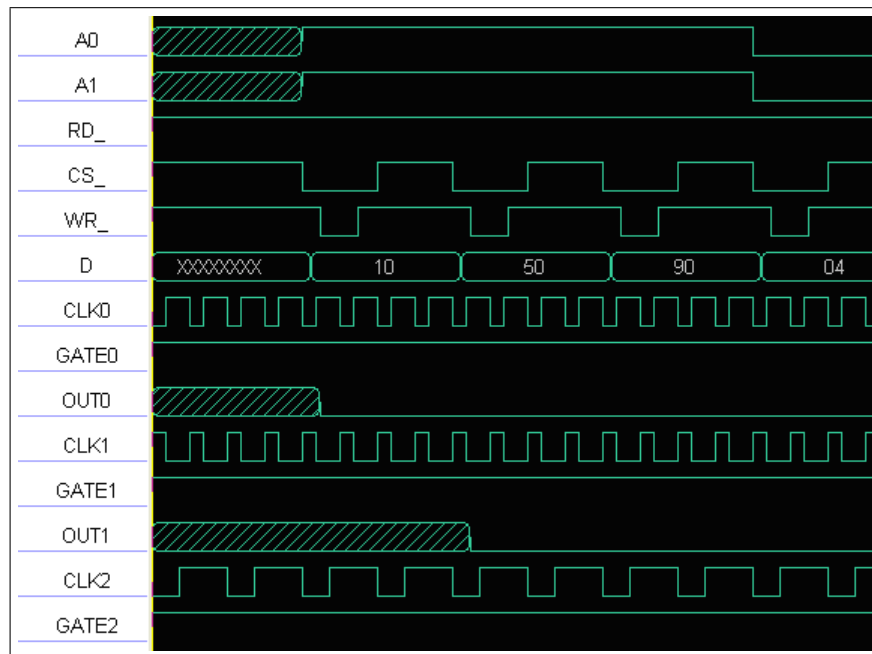


Figure 1.1: Graphical representation of event based data

On the other hand, Automated Test Equipment (ATE) requires a cycle based definition for hardware operation. The reasons are basically related to hardware limitations (not relevant in software simulators) due to synchronization related issues. Another reason for the need to cyclize the event stream data is to enable timing specification characterization. Timing specifications are defined within the device data sheet and should be measured using the ATE. In order to achieve this, a cycle which implements the timing relations should be identified, and the ATE needs to measure the timing ratio. The device cycle used in ATE test vectors should match the device operation mode. An ATE compliant cycle is typically limited to the number of events per cycle. An example of tester cycle is shown in Figure 1.2.

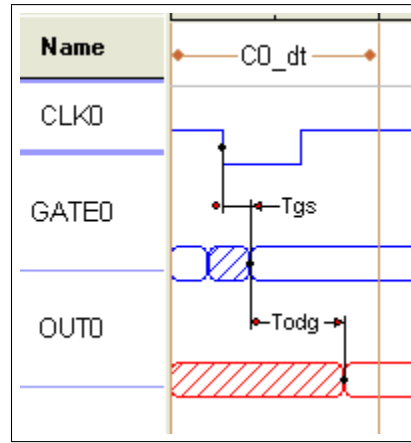


Figure 1.2: Example of tester cycle

When designing a device, simulation is used to emulate the device activity. The test trace generated by the simulator contains events of the device signals. Functional simulation traces DUT signals activity and records all events in formats designed to record such data. Event data format records signal activity as a long stream of signal events. With ATE (Automatic Test Equipment) it is currently impossible to execute such a test sequence. ATE uses cycle format, resynchronizing all signals within a certain period.

The project is started when the device specification is defined. At this stage, the device designers draw timing diagrams that describe its operation. Then, designers typically develop patterns to simulate the device and verify that the device model responds as defined. At this point, starts the main problem of preparing the test program: simulators used for design stage are event driven, testers used to verify the actual silicon functionality after production are cycle-driven.

VCDSTIL is designed to translate VCD and EVCD event-based test formats into STIL cycle-driven format, therefore providing a path from simulation results to industry standard test description language.

STIL generated by VCDSTIL can then be translated into a test program for various ATE's using the Verigy ATEGen tool. This provides a complete flow from simulation results to an ATE test program. See the ATEGen user manual for more information.

Since the design verification stage does not take into account production tester constraints, test engineers often need to adjust the simulation results to make those tester compliant. To handle this aspect, another relevant tool is the Verigy VCD Preprocessor which can be used to manipulate VCD and EVCD formatted tests prior to translation into STIL. Possible operations include cutting the tests, shifting signal edges, changing signal values, creation of virtual signals, etc. See the VCD Preprocessor User Manual for more information.

Chapter 2

Installing VCDSTIL

VCDSTIL is part of the Verigy STDL Toolset and is installed with other STDL Toolset applications. For more information, see the STDL Toolset Installation Guide.

Chapter 3

VCDSTIL Operational Principles

3.1 Cyclization Of Event Based Data

Production testers require that all signal waveforms are defined in every cycle during the test. VCDSTIL needs to read a given event driven formatted test and translate it into a cycle-based Standard Test Interface Language (STIL), within which every signal is assigned a pre-defined waveform in every cycle.

The VCDSTIL methodology offers a flexible approach to cycle matching. It uses the independent definition of the two elements in cycle matching:

- The condition defining *when* (within the event stream) a specific cycle is to be used.
- The cycle definition of *how* the cycle should be implemented using the ATE hardware (which may or may not be similar to the event stream). Part of the cycle definition may include a point where the signal state is sampled from the event stream data.

By separating these two elements, the proposed methodology addresses the issue that simulation does not follow the exact device cycle specification definition. The selection of a specific cycle is done based on the pre-defined condition, which may include any signal in the simulation.

Users can include an artificial indicator of specific device modes to control cycle selection and thus the timing spec selection. The selection is activated on a signal or a group of signals independently. This greatly simplifies the permutations among signals per the different device modes. Section [3.4 VCDSTIL Cycle Matching](#) describes the specific modes used.

The tool:

- Reads the event driven data.
- Splits the whole sequence into cycle periods pre-defined by the user.
- It then goes through every signal in the list and checks which waveform should be assigned in the current cycle.
- It verifies the conditions, and the first condition matched will assign its corresponding waveform to the signal in the current cycle.

- If no-condition is matched in any given cycle, the tool will stop and issue an error.
- When finished assigning a waveform throughout the test, the tool writes the resulting STIL file and quits.

3.2 Understanding Event Based Data

The event based data is represented by value change dump files (VCD and EVCD formats). Both formats are IEEE standard ones, 1364-1995 and 1364-2001 correspondingly. The data in VCD and EVCD files represents a trace of logic simulation and provides information about the values for each signal at any given time point in the simulation.

3.2.1 VCD Files

There are four states that represent logic values of signals in a VCD file. Signal direction information is not available in VCD. Additional means are used to specify signal direction for VCDSTIL, for example control pins.

Value	Description
0	Logic low
1	Logic high
Z or z	Tristate
X or x	Unknown

Table 3.1: VCD logic values

3.2.2 EVCD Files

Signal states in EVCD files already include direction information.

3.3 Understanding STIL

STIL is an IEEE standard 1450 for IC test vectors which many automatic test program generator tools (e.g. TetraMAX, FastScan, Encounter Test) use for their results. STIL is used by VCDSTIL as a generic output that can be translated further to ATE syntax. For more information, see IEEE Standard Test Interface Language (STIL) for Digital Test Vector Data (IEEE Std 1450-1999).

STIL files consist of several blocks that define device signals, signal groups, timing and vectors. The following is an example of a STIL file that defines two signals, one timing table and a test pattern consisting of three vectors.

Value	Description
Input	
D	Drive zero
U	Drive high
N	Drive unknown
Z	Tri-state
d	Drive zero (two or more drivers active)
u	Drive high (two or more drivers active)
Output	
L	Compare to logic low
H	Compare to logic high
X	Unknown (don't care)
T	Compare to midband
l	Compare to logic low (two or more drivers active)
h	Compare to logic high (two or more drivers active)
Unknown direction	
0	Logic low (both input and output are active with 0 value)
1	Logic high (both input and output are active with 1 value)
?	Unknown
F	Tri-state (input and output disconnected)
A	Unknown (input 0 and output 1)
a	Unknown (input 0 and output X)
B	Unknown (input 1 and output 0)
b	Unknown (input 1 and output X)
C	Unknown (input X and output 0)
c	Unknown (input X and output 1)
f	Unknown (input and output tri-stated)

Table 3.2: EVCD logic values

```

1  STIL 1.0;
2
3  Signals {
4      clk In;
5      data InOut;
6  }
7
8  SignalGroups {
9      all = 'clk + data';
10 }
11
12 Timing {
13     WaveformTable t1 {
14         Period '100ns';
15         Waveforms {
16             clk      { P0 { '0ns' D; '25ns' U/D; '75ns' D; } }
17             data      { 01 { '50ns' D/U; } }
18             data      { LHX { '0ns' Z; '50ns' L/H/X; } }
19         }
20     }
21 }
22
23 PatternBurst B1 {
24     PatList {
25         P1;
26     }
27 }
28
29 PatternExec PE1 {
30     PatternBurst B1;
31 }
32
33 Pattern P1 {

```

3.4 VCDSTIL Cycle Matching

The timing file used by VCDSTIL for cycle matching consists of the following elements defined for each signal:

- Waveform character.
- List of waveform definitions.
- Condition for each waveform.

The following figure is an example of a timing file with different cycle matching options.

Pin	WFC	Waveform	Condition
A1	U	100ns:U	waveform
A1	D	100ns:D	waveform
CLK0	D	0ns:D	(D_dir=='U')@(t0)
CLK0	P	100ns:D; 250ns:U	waveform
CLK1	U	100ns:U; 250ns:D	true

Figure 3.1: Timing definition with different matching options

3.4.1 Force User-Defined Timing

Specifying **true** in the **Condition** column of a timing spreadsheet enables you to assign the associated waveform regardless of actual signal behavior in the simulation.

In the example above, the CLK1 signal is always assigned a waveform that drives one at 100ns and changes to zero at 250ns from the beginning of the cycle regardless of the actual behavior of the signal in the simulation.

3.4.2 Condition Based Waveform Matching

Using conditions enables you to assign the associated waveform only when the condition specified is met. The condition consists of signal states at specific timepoints, both inside and outside cycle boundaries.

In the example above, the first waveform of the CLK0 signal is assigned by VCDSTIL when the value of the D_dir signal equals 1 (U means "input high") at the beginning of the cycle (specified by @(t0)).

3.4.3 Waveform Matching

This mode of operation means that the waveform defined by you is also the condition. This means that the waveform is assigned if this waveform matches the waveform in the event driven data. The tool compares the waveform definition to the event stream, and, if the two match, the defined cycle is used for that portion of the test.

In the example above, the second waveform of the CLK0 signal is assigned when the signal in the simulation file drives zero at 100ns and changes to one at 250ns from the beginning of the cycle (waveform is specified as 100ns D; 250ns U;).

3.4.4 How Waveforms Are Mapped To STIL

The waveforms defined in the timing file are a variation of STIL waveforms. A STIL waveform is a sequence of edges (zero or more) at specific timing points, which describe the desired signal behavior during a given tester cycle. Figure 3.2 demonstrates the use of different VCDSTIL waveforms.

Pin	WFC	Waveform	Condition
A0		Tper*0.25:F	true
A1	U	100ns:U	waveform
A1	D	100ns:D	waveform
CLK0	D	0ns:D	(D_dir=='U')@(t0)
CLK0	P	100ns:D; 250ns:U	waveform
CLK1	U	100ns:U; 250ns:D	true
OUT1		Tper*0.75:R	true
DATA		Tper*0.5:S@(Tper*0.75)	true

Figure 3.2: Example of VCDSTIL waveforms

Table 3.3 demonstrates how specific VCDSTIL waveforms are mapped to STIL waveforms. This table is *not* a part of the VCDSTIL timing file and is provided for explanation only.

Table 3.4 describes the subset of STIL edges in all cycle matching modes supported by VCDSTIL.

In order to enable VCDSTIL to ignore portions of event based data an additional set of edges is supported in waveform matching mode. Table 3.5 describes these edges.

Additional edges are supported in user-defined timing and condition based modes. In both modes, it is possible to use special edges that select data from the simulation. The latter edges are an extension of VCDSTIL and do not exist in the STIL standard. Table 3.6 describes these edges.

3.5 How STIL Pattern Data (WFC) Is Formed

The STIL pattern data (called Waveform Character or WFC in the STIL standard) can be assigned either by you or automatically by VCDSTIL. You can assign pattern data only if edges that select data from the simulation (see Table 3.6) are not used. See the *WFC* column in Figure 3.1 for an example of user-defined and automatically assigned pattern data.

VCDSTIL Waveform	STIL Waveforms	Description
100ns:D	100ns:D;	Drive zero at 100ns from the beginning of the tester cycle.
100ns:U	100ns:U;	Drive high at 100ns from the beginning of the tester cycle.
100ns:D; 250ns:U	100ns:D; 250ns:U;	Clock pulse with negative edge at 100ns and positive edge at 250ns.
Tper*0.25:F	Tper*0.25 D; Tper*0.25 U;	Edge F is a shortcut for "drive data". The value is selected from the simulation at Tper*0.25. D is used if the simulation data is 0 and U is used if the simulation data is 1.
Tper*0.75:R	Tper*0.75 L; Tper*0.75 H; Tper*0.75 X; Tper*0.75 T;	Edge R is a shortcut for "compare data". The value is selected from the simulation at Tper*0.75. Edges L, H, X and T are used when the simulation data is compared with low, high, unknown or midband used respectively.
Tper*0.5:F@(Tper*0.75)	Tper*0.5 D; Tper*0.5 U;	In this waveform, the data to be driven is taken from Tper*75, even though it is applied at Tper*0.5.

Table 3.3: Mapping timing waveforms to STIL

Timing Edge	Description
D	Drive zero
U	Drive high
N	Drive unknown
P	Drive previous
Z	Turn driver off
L	Compare to logic low
H	Compare to logic high
X	Compare to unknown
T	Compare to value between logic high and logic low (mid-band)

Table 3.4: STIL timing edges supported by VCDSTIL

Mask Edge	Description	Example
MaskAny,a	Mask all events from this edge until the next edge in the cycle or cycle end	{ 0ns:MaskAny; 50ns:H; }
MaskInput,i	Mask all input events from this edge until the next edge in the cycle or cycle end	{ 0ns:D; 20ns:MaskAny; 70ns:U; }
MaskOutput,o	Mask all output events from this edge until the next edge in the cycle or cycle end	{ 0ns:MaskOutput; 70ns:L; }

Table 3.5: Mask edges supported by VCDSTIL in waveform matching mode

Timing Edge	Description
F	Drive data (low or high) depending on actual data in the simulation. Signal direction must be input for this edge or an error will be generated.
R	Compare data (low, high, unknown or midband) depending on the actual data in the simulation. Signal direction must be output for this edge or an error will be generated.
S	Drive or compare data, depending on actual data and signal direction in the simulation.

Table 3.6: Special timing edges supported by VCDSTIL in user-defined and condition based waveforms

Chapter 4

Using VCDSTIL

4.1 Introduction

VCDSTIL operates via a command line interface, controlled by a setup file. VCD and EVCD event based formats are converted into STIL cycle driven format files, according to the settings in the setup file. In this setup file, you define the following main setup parameters and inputs:

- Paths to the following files in:
 - input (optional, can be also specified from command line).
 - timing (mandatory).
 - pins definition (mandatory).
 - pins group (optional).
- Cycle width (mandatory).
- List of signals to be included in the output STIL file (optional. By default all signals are selected).

4.2 Setting Up Signals

VCDSTIL converts an event-based simulation file into a cyclized pattern test for a tester.

In the simulation file, a signal's direction is unknown. This means that the test engineer must specify the direction of the pin in VCDSTIL (in the Pins definition file). VCDSTIL uses this information to decide whether the tester should drive or compare data. There are three types of pin direction: input, output, and input/output (I/O). When a pin direction is input or output, it means that this direction stays the same during the functional/structural test. When a pin direction is I/O it means that this pin is bidirectional; its direction may change during the test. VCDSTIL needs to know when this pin is input and when it is output.

VCDSTIL uses an additional internal signal to identify I/O pin direction. This internal signal should be included in the simulation file. This signal is the I/O control pin. You have to define, for each inout pin, its control signal. This is done by entering it in the I/O Control Pin property in the CSV file. See [Direction Controls](#) for more details.

4.2.1 Import Signals from Event Based Files

In order to import the existent list of signals from the VCD/EVCD file automatically into the pins definition file, you need to run VCDSTIL with the following flag,

--> `-work_mode: gen_pins`, which means run in pins generation mode.

Run command example:

```
vcdstil -work_mode: gen_pins -setup MySetup.py
```

The pin list is imported to the pins definition file, defined in the setup file in the following parameter:

```
pins_file = 'Pins.csv'
```

The pin list contains only pins that are defined in the imported file. The direction of the pins is set to in if the input file is VCD, and inout in case of EVCD.

4.2.2 Updating Signal Properties

When importing signals from a VCD file, only the names of the pins are extracted. Other signal properties, such as pin direction, are set to defaults (in or inout) and need to be modified by the user.

	A	B	C	D	E	
1	<VS Pins>	1.0				
2	Pin	Type	I/O Control	State Map	SSP	
3	D_dir	in			0	
4	A0	in			0	
5	A1	in			0	
6	CLK0	in			0	
7	CLK1	in			0	
8	CLK2	in			0	
9	CS_	in			0	
10	D0	in			0	
11	D1	in			0	
12	D2	in			0	
13	D3	in			0	
14	D4	in			0	
15	D5	in			0	
16	D6	in			0	
17	D7	in			0	
18	GATE0	in			0	
19	GATE1	in			0	
20	GATE2	in			0	
21	OUT0	in			0	
22	OUT1	in			0	
23	OUT2	in			0	
24	RD_	in			0	
25	WR_	in			0	
26	</VS Pins>					

Figure 4.1: Example of Imported Pins File

The Pins definition file contains the following fields:

- **Pin:** Specifies the Pin name taken from the imported file. Mandatory.
- **Type:** Specifies the pin direction, which can be one of the following: in, out, inout. Mandatory. When importing signals from a (E)VCD file, pin direction is set to default values and should be modified by the user.
- **I/O Control:** Controls the direction of the pin. This field is only valid for VCD inout pins for which it is mandatory.
- **State Map:** Controls translation of (E)VCD states to STIL states. This advanced feature defines a mapping between simulation states and output STIL states. See ?? for information about using state maps.
Note that the state map has to cover all possible combinations of states. Use of wildcard "." (stands for "every state") is allowed. The wildcard is useful to cover all states that were not covered by other map entries.
- **SSP:** Enables (1) or disables (0, the default) of Smart Strobe Positioning (SSP). The purpose of SSP is to minimize the number of output waveforms and strobe positions created due to output waveform phases. For each signal, VCDSTIL tries to locate a common strobe position for all output waveforms of that signal.

Direction Controls

Direction control enables you to specify when a VCD inout signal's value should be translated to the input state and when to the output state, based on current values of one or two control signals.

The format of the direction control definition is:

"in:pin=states,out:pin=states".

The following are examples of signal attributes with direction control defined:

When D_dir = 0 (Low) signal D[7] direction is input

When D_dir = 1 (Up) signal D[7] direction is output

```
1 "D[7]"          "inout"          "in:D_dir=0,out:D_dir=1"
```

The first definition means that when the value of the D_dir signal is 0, all values of D[7] are translated to input states (0 to D, 1 to U, etc.) when the value of D_dir is 1, all values of D[7] are translated to output states (0 to L, 1 to H, etc.).

	A	B	C	D	E
1	<VS Pins>	1.0			
2	Pin	Type	I/O Control	State Map	SSP
3	D0	inout		[D_Dir, D0] 10->D ?X->X 0Z->X 01->H 00->L	0
4	D1	inout	in:D_dir=0,out:D_dir=1		0
5	D2	inout	in:D_dir=0,out:D_dir=1		0
6	D3	inout	in:D_dir=0,out:D_dir=1		0
7	D4	inout	in:D_dir=0,out:D_dir=1		0
8	D5	inout	in:D_dir=0,out:D_dir=1		0
9	D6	inout	in:D_dir=0,out:D_dir=1		0
10	D7	inout	in:D_dir=0,out:D_dir=1		0
11	GATE0	in			0
12	GATE1	in			0
13	GATE2	in			0
14	OUT0	in			0
15	OUT1	out			1
16	OUT2	out			1
17	RD_	in			0
18	WR_	in			0
19	</VS Pins>				

Figure 4.2: Example of Pins Definition File with Direction Controls

4.3 Settings Up Groups

Defining the pins into groups is performed by defining the groups in the Pins Group file. This file is defined in the setup file in the following parameter:

```
groups_file = 'groups.csv'
```

The Pins Group file contains the following fields:

- **Group** - Pin Group name (mandatory).
- **Pins** - Group members wildcards (?*) are allowed (mandatory).

The following is an example of a Pin groups file where:

- Group **ADDR** includes [A0, A1, A2, A3] signals
- Group **DBUS** includes [D0, D1, D2, D3, D4, D5, D6, D7] signals

	A	B
1	VS Groups	1.0
2	Group	Pins
3	ADDR	A0 A1 A2 A3
4	DBUS	D0 D1 D2 D3 D4 D5 D6 D7
5	<VS Groups	

Figure 4.3: Example of Pins Group File

4.4 Defining Signal Timing

The timing definition for each of the signals is done using a timing file. VCDSTIL uses the timing file specified in the setup file in the following parameter:

`timing_file = 'Timing.csv'` timing definitions file

4.4.1 VCDSTIL Waveform Definition

The timing file contains the following fields:

	A	B	C	D
1	VS Timing	1.0		
2	Pin	WFC	Waveform	Condition
3				
4	<VS Timing			

Figure 4.4: Example of an Empty Timing File

- **Pin** - Pin name (mandatory).
- **WFC** - Waveform character (optional).
- **Waveform** - Waveform description (mandatory).
- **Condition** - Waveform matching condition (mandatory).

Each line defines a waveform for the pin specified in the first column. During STIL generation, VCDSTIL tries to match pin waveforms, in order of definition. The way of matching a specific waveform depends on its condition. If matching of one waveform fails, VCDSTIL attempts to apply the next one. If all waveforms defined for the specific pin cannot be matched, an error is generated.

There are three types of waveform matching:

- For matched waveforms, where the waveform is also the condition, the condition field contains the following keyword:
'waveform'.
This means that during STIL generation, the waveform events are compared to the VCD events of the current cycle. If the waveform matches the VCD contents, it is applied.
- For forced waveforms, the condition field contains the keyword:
'true'.
These waveforms always match. A separation between input and output cycles is supported by use of **F/R** events.
- Conditional waveforms have the actual condition expression defined. VCDSTIL checks the condition and applies the waveform if, and only if, the condition is resolved to true.

The **WFC** field defines an ASCII character to use as a waveform character in STIL timing. If WFC is not specified, VCDSTIL selects a STIL waveform character automatically, based on waveform events. WFC cannot be defined for waveforms featuring **S/F/R** events (see below).

The **Waveform** field defines waveform events, separated by a semicolon, e.g., **0ns:D; 10ns:U; 20ns:D**. Each event consists of a time and an action. Time is an expression that can include constants as well as specification parameters and equations.

The following actions are supported:

- D: Driver on, drive low.
- U: Driver on, drive high.
- N: Driver on, drive high.
- Z: Driver off.
- L: Compare low.
- H: Compare high.
- T: Compare midband.
- X: Compare off.
- P: Drive prior (supported only in forced/conditional waveforms).
- S: Use pattern data (supported only in forced/conditional waveforms).
- F: Use pattern data, input only (supported only in forced/conditional waveforms).
- R: Use pattern data, output only (supported only in forced/conditional waveforms).

Pin	WFC	Waveform	Condition
D0		8ns:D; tcyc*0.6:S(t0-2ns)	S1 == '0' and S2 != '1'
D0		"tclk+tpd":S	waveform
D0		"tclk+tpd*2":S	waveform
D0		75ns:D	waveform
D1	P	10ns:D; 40ns:D; 70ns:U	true

Figure 4.5: Example of a Timing File with Waveforms and Conditions

4.4.2 Extracting Timing From Event Based files

To automatically extract timing from (E)VCD, run VCDSTIL with the following flag:

`-work_mode: gen_timing`

Before running VCDSTIL with the `gen_timing` flag, the following flags must be defined in the setup file: pins definition file, timing file, input files, cycle width and active pins

```

1 pins_file    = 'Pins.csv'      # pins definitions file (Mandatory)
2 timing_file  = 'Timing.csv'    # link to the extracted timing file (Mandatory with gen_timing)
3 input_files  = ['sample.vcd']  # input vcd/evcd files list (Mandatory)
4 cycle_width  = '20ns'         # cycle width (Mandatory)
5 active_pins  = []             # list of active pins, default is select all pins from Pins.csv

```

Run command example:

`vcdstil -setup MySetup.py -work_mode: gen_timing`

The pins timing is automatically extracted to the timing file defined in the setup file in the following parameter:

`timing_file = 'Timing.csv'`

	A	B	C	D
1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	test1.U.CLK0	U	0ps:U; 250000ps:D	waveform
4	test1.U.CLK1	D	0ps:D; 250000ps:U	waveform
5	test1.U.D0	L	350000ps:Z; 350000ps:L	waveform
6	test1.U.D0	H	350000ps:Z; 350000ps:H	waveform
7	test1.U.GATE0	U	250000ps:U	waveform
8	test1.U.GATE0	D	250000ps:D	waveform
9	test1.U.GATE0	u	250000ps:U	waveform
10	test1.U.OUT0	L	50000ps:Z; 50000ps:L	waveform
11	test1.U.OUT0	H	250000ps:Z; 250000ps:H	waveform
12	test1.U.WR_	D	50000ps:D	waveform
13	test1.U.WR_	U	50000ps:U	waveform
14	</VS Timing>			

Figure 4.6: Example of Timing File with Extracted Waveforms

4.4.3 VCDSTIL Forced Waveforms

For forced waveforms, the condition field contains the keyword **'true'**. These waveforms are used/assigned by VCDSTIL unconditionally. VCDSTIL does not check a condition before assignment. The only exception is the signal direction, which is checked by VCDSTIL.

Forced waveforms support the special actions **S/F/R**. Instead of defining a specific pin behavior, these actions mean that the (E)VCD state is sampled and substituted in the waveform. **S** actions match every (E)VCD state; **F** actions only match input states; **R** actions only match output states. Therefore, using **F/R** actions enables you to define separate forced waveforms for input and output cycles. In output cycles, a waveform with **F** is not applied. Similarly, for input cycles, a waveform with **R** is skipped.

For **S/F/R** events, in addition to time and action, a sample point can be provided, e.g., **10ns:S(20ns)**. Sample points instruct VCDSTIL to sample pattern data in the specific point of the current cycle. In this example, VCD state is sampled at 20 ns from the cycle start. By default, if a sample point is not specified, event time is used (10 ns in the example).

Waveforms with **S/F/R** actions can generate multiple STIL waveforms. For example, waveform **0ns:F** can result in the generation of **0ns:D** or **0ns:U** waveforms depending on the VCD sampled data in the resulting STIL. Consequently, WFC cannot be defined for these waveforms.

	A	B	C	D
1	VS Timing	1.0		
2	Pin	WFC	Waveform	Condition
3	OUT		90ns:R	true
4	IO[1]		80ns:R	true
5	IO[1]		20ns:F	true
6	TCK		50ns:F	true
7	<VS Timing			

Figure 4.7: Example of a Timing File with Forced Waveforms

Expression	Description
signal == 'states'	Signal equals one of the states
signal != 'states'	Signal is not equal one of the states
signal == 'states' [@check-point]	Signal equals one of the states at specified checkpoint
signal1 == 'states' [@check-point] and signal2 == 'states' [@check-point]	Complex exprssion that checks that both signal1 and signal2 equals on of the specified states
signal == 'states' [@check-point] or signal2 == 'states' [@check-point]	Complex exprssion that checks that either signal1 and signal2 equals on of the specified states
signal == stable(start,length,ignore direction change)	Complex expression that check that the signal is stable in the specified range

Table 4.1: Available Condition Syntax

4.4.4 VCDSTIL Conditional Waveforms

In addition to the separation between input and output cycles, forced waveforms have a mechanism which controls the selection of waveforms for a specific pattern cycle. When such a mechanism is needed, conditional waveforms should be used. They differ from forced ones in having a real conditional expression in the condition column. VCDSTIL checks the condition, and applies the waveform if, and only if, the condition is resolved to true.

Conditional expressions consist of :
signals, signal value and time.

states:

VCD/EVCD valid states. You are able to use a range of possible states for single-bit state.

A List of valid states' values for the use in condition:

D,U,N,Z,L,H,T,X.

In addition, hexadecimal values can be used to make bus-checking conditions more compact. Hexadecimal values should have a prefix of 'xi:' (in this case they will be compared to input states) or 'xo:' (for outputs).

For example, "BUS=='xi:F0'@(t0)" is equivalent to "BUS=='xi:UUUDDDD'@(t0)".

check-point:

Checkpoint defines the exact location in the simulation flow where the condition is evaluated.

VCDSTIL checks the values of the specified signal at the specified time point, and compares the result to the value specified by the user.

For example, if the waveform has a condition **D_dir=='U'@(t0+5ns)**, the waveform is only applied if signal **D_dir** has a state **U** at 5 ns from cycle start (t0 stands for current cycle start).

It is possible to combine multiple checks, using AND/OR logical operations, for example **D_dir=='U'@(t0) and CLK=='D'@(t0)**.

Stable:

A special type of condition checks the stability of the signal value in a specified interval. The format of the condition is:

signal==stable(start,length,ignore direction change).

It accepts three parameters:

- Start of the interval
- Length of the interval
- Optional "ignore direction change" flag (default is 0). If the flag is set to 1, signal value changes are ignored as long as they are only direction changes (e.g., input high to output high).

For example, the condition **SIG0==stable(t0+10ns,20ns)** is only true if the signal **SIG0** does not change value between 10ns and 30ns from cycle start.

Examples:

- **(S1 == '1') @(t0-5ns) and (S3 == 'X')**
True when S1= '1' 5ns before current cycle start (t0) , and S3 = 'X' at cycle start.
- **(Data == '0010??')@t0**
True when data starts with '0010' at current cycle start (t0). The 2 last bits are ignored, but the overall state size must be 6.

4.4.5 Multiport Mode

VCDSTIL enables you to split a signal set into a number of 'ports', that is groups of signals which may have a cycle width different from the cycle width of other ports. For each port, a separate STIL Waveform Table and a separate pattern are generated.

The ports file is a csv sheet with the following fields:

- **Port** - Port name (mandatory).
- **Pins** - Port pin list (mandatory).
- **Period** - Port cycle unit (mandatory).

The Port file is defined in the setup file in the following parameter:

```
ports_file = 'Ports.csv'
```

The following is an example of a Port file, where:

- Port1 includes [Clk1, D0, D1, D2] signals, cycle unit 30ns
- Port2 includes [Clk2, D3, D4, D5] signals, cycle unit 50ns

	A	B	C
1	VS Ports	1.0	
2	Port	Pins	Period
3	Port 1	Clk1 D0 D1 D2	30ns
4	Port 2	Clk2 D3 D4 D5	50ns
5	<VS Ports		
6			

Figure 4.8: Example of a Port File

4.5 Settings Up AC Specs

Spec variables used in the equations are initialized in an external STIL standard file.

This file is defined in the setup file in the following parameter:

```
specs.file = 'Specs.stil'
```

The following is an example where two specs parameters. **Period** and **sclk_offset** are defined in the timing file and how they are set up in the specs file:

VCDSTIL imports spec data from the external specs file.

Timing file content example:

	A	B	C	D
1	VS Timing	1.0		
2	Pin	WFC	Waveform	Condition
3	clkin		0*Period+sclk_offset.P;	true
4	<VS Timing			

Figure 4.9: Example of a Timing File with AC Specs Period and sclk_offset

Spec STIL file content example:

```

1  STIL 1.0 {
2    Design 2005;
3  }
4  Spec {
5    Category func {
6      Period      { Typ '64ns'; Min '0ns'; Max '0ns'; }
7      sclk_offset { Typ '1ns';  Min '0ns'; Max '0ns'; }

```

4.6 Setting Up the VCDSTIL Setup File

4.6.1 Creating VCDSTIL Setup Template Files

VCDSTIL enables you to create a setup template for csv files and the main VCDSTIL setup file with all of the available operations and commands that you can select and edit, in order to run VCDSTIL.

Using this file enables you to include signals and their attributes, such as type, direction controls, predefined timing waveforms and state maps.

The VCDSTIL setup file enables you to control a wide range of aspects of the conversion process.

Create a setup template file

In order to generate a CSV template and the main VCDSTIL setup file, use the `-template` command line parameter:

For example `> vcdstil -template`

When running this command, VCDSTIL automatically generates all template files in `./templates` directory and silently exits.

Open the files, add and edit options as desired.

For additional information about the available options, see the [VCDSTIL setup file](#).

Save the edited setup file.

There is no standard file name extension for setup files (although `.setup` and `.py` are common). No special location is required for the file; you can save it in any convenient directory.

Result

You can now use the setup file by specifying it on the command line using the `-setup` option for example.

```
vcdstil -setup ./MySetup.py
```


4.6.2 Setup File Parameters

This section describes all VCDSTIL setup file parameters and commands.

Below is the list of setup file parameters.

External Files:

- **pins_file**
Example: `pins_file = 'Pins_direction_setup.csv'`
Specifies the pins direction definitions file.
The parameter is mandatory.
- **timing_file**
Example: `timing_file = 'mydir/mytiming.csv'`
Specifies the signals timing definition file.
The parameter is mandatory.
- **groups_file**
Example: `groups_file = 'mydir/mygroups.csv'`
Specifies the signals groups definitions.
The parameter is optional.
- **ports_file**
Example: `report_file = 'mydir/ports.csv'`
Specifies the ports definitions.
The parameter is optional.
- **transform_file**
Example: `transform_file = 'mydir/transform.csv'`
Specifies the transformations definitions.
The parameter is optional.
- **wftmap_file**
Example: `transform_file = 'mydir/wft.csv'`
Specifies the WFT map definitions file.
The parameter is optional.
- **specs_file**
Example: `specs_file = './mydir/specs.stil'`
Specifies the specs definitions file in STIL format.
The parameter is optional.

Conversion Attributes:

- **input_files**

Example: `input_files = [ 'f1.evcd', 'f2.evcd' ]`

Specifies the input vcd/evcd files list. The parameter is mandatory.

- **stil_out_file**

Example: `stil_out_file = 'mydir/myfile.stil'`

Specifies the STIL output file name.

- **start_time**

Example: `start_time = '1000ns'`

Extraction start point. Useful when part of the (E)VCD should be ignored.

Optional. Default is 0.

- **end_time**

Example: `end_time = '2000ns'`

Extraction end point. Useful when part of the (E)VCD should be ignored.

Optional. Default is the last time point of (E)VCD file.

- **cycle_width**

Example: `cycle_width = '20ns'`

Cycle width. Mandatory.

- **active_pins**

Example: `active_pins = ['A','B','C']`

List of pins to be included in the output stil file. The parameter is optional (default = select all).

Pin Names Extraction:

- **keep_design_hierarchy**

Example: `keep_design_hierarchy = 1`

If set to 1, the full hierarchical path of a signal is used as the signal name that is, signals will have names like ".mymodule1.mymodule2.mysig". If set to 0, only the signal name is stored (i.e., the signal will be named "mysig"). This parameter is optional. (default is 1)

- **select_design_modules = []**

Example: `select_design_modules = ['module2', 'module4']`

The modules in which to process pins. This parameter is optional. (default = select all)

- **keep_design_hierarchy_for_pins = []**

Example: `keep_design_hierarchy_for_pins = ['A', 'C', 'D']`

Keep full design path only for a limited set of pins, even if `keep_design_hierarchy` is set to 0. This parameter is optional. (default = [])

Global:

- **vcd_bus_order**

Example: `vcd_bus_order = 'ascending'` This is the default order that the tool assigns to the (E)VCD buses, either 'ascending' [0:n-1] or 'descending' [n-1:0]. This parameter is optional. (default = ascending)

- **max_memory_allocation** Example: `max_memory_allocation = 1024`

The maximum memory allocation (in megabytes) to be used by the tool. (default = 1024)

- **max_spike_width** Example: `max_spike_width = '2ns'` The maximum spike width to be ignored by the tool. (default = 0ns)

- **time_unit** Example: `time_unit = 'ps'`

The default time unit used for equations [fs — ps — ns — us — ms — s] (default = ps)

- **generate_timestamps**

Example: `generate_timestamps = 0`

Generate time point comments for pattern vectors in the output STIL file. (default = 0))

- **stop_on_fail**

Example: `stop_on_fail = 1`

Stop if cycle matching fails.

(default = 1, stop after the first time the cycle matching fails)

- **interim_vcd**
Example: `interim_vcd = 'try.vcd'`
interim vcd file path (interim vcd is not written if the path is empty)
Creates the vcd file with debug information.
- **auto_generate_wft** Example: `auto_generate_wft = 0`
This parameter is set to 1 for auto generate WFT, when non of the WFT specified in the WFT map file is matched.
(default = 0, to abort with error)
- **max_pin_waveforms**
Example: `max_pin_waveforms = 10`
Specifies the limit for a number of generated waveforms per signal. This value must be in the range 1..62.
This parameter is optional. (default = 62)
- **max_vectors**
Example: `max_vectors = 1000`
Defines the maximum number of vectors allowed per pattern. This parameter is optional. (default = -1, number of vectors is unlimited)
- **allow_new_waveform**
Example: `allow_new_waveform = 1`
If set to 0, the automatic generation of waveforms is disabled. VCDSTIL stops when a waveform extracted from the (E)VCD does not fit the master file and predefined timing waveforms. This parameter is optional. (default = 1)

STIL Writer Customization This section describes STIL writer customization options

- **stil_file_layout**

Example: `stil_file_layout = 'normal'`

The layout of the STIL files. This parameter is optional. (default = 'normal')

Valid values are:

- 'normal': top file = [stil_project_name].stil pattern = [stil_project_name]/[pattern_name].pat.stil
- 'file_per_block':
 - top file = [stil_project_name].stil
 - signals = [stil_project_name].sig.stil
 - groups = [stil_project_name].grp.stil
 - dlevels = [stil_project_name].dc.stil
 - scan = [stil_project_name].scan.stil
 - specs = [stil_project_name].spec.stil
 - timing = [stil_project_name].tim.stil
 - env = [stil_project_name].env.stil
 - bursts = [stil_project_name].brst.stil
 - execs = [stil_project_name].exec.stil
 - pattern = [stil_project_name]/[pattern_name].pat.stil
- 'all_in_one':
 - file = [stil_project_name].stil

- **stil_pattern_mode**

Example: `stil_pattern_mode = 'write_on_change'`

The STIL pattern formatting mode. This parameter is optional. (default = 'write_on_change')

Valid values are:

- 'write_on_change':
 - V { Sig1 = D; Sig2 = U; Sig3 = 'H'; }
 - V { Sig1 = U; }
 - V { Sig3 = X; }
- 'write_as_group':
 - V { _ALL_ = DUH; }
 - V { _ALL_ = UUH; }
 - V { _ALL_ = UUX; }

- **stil_detect_repeats**

Example: `stil_detect_repeats = 0`

Detect repeated vectors. Transform repeated identical vectors to STIL Loop statements. This parameter is optional. (default = 0)

- **stil_comment_type**

Example: `stil_comment_type = 'C++'`

STIL comment format type. This parameter is optional. (default = 'C++')

Valid values are: 'C++', 'C'

Defines a mapping between simulation states and output STIL states. See [for](#) information about using state maps.

- name: Unique map ID. Mandatory.
- inherit: Inherit from another map. Optional. Inherited mapping can be overridden.
- dim: Map dimension, defines a number of input states. Mandatory.
- data: Mapping definition. Length of strings in the left part must match dimension attribute. Wildcard (.) is allowed in the left part. Mandatory.

4.7 Using Command Line Interface (CLI) of VCDSTIL

The VCDSTIL can be controlled by command line parameters, setup file parameters or a combination of both. The command line parameters have higher priority than setup file parameters, meaning the latter may be overridden using command line parameters.

4.7.1 Command Line Options Description

Synopsis:

```
vcdstil [-help] [-version] [-timestamp] [-licwait] [-template] [-setup setup-file ]
        [-stil_out_file:<str>] [-work_mode:<str>] [(E)VCD-input-files]
```

The following are the available command line options for VCDSTIL:

-help	Display help message and exit.
-version	Show version number and exit.
-timestamp	Enable log timestamp.
-licwait	Wait for the license, if not available. The application will wait indefinitely until a license becomes available.
-progress	Show the progress bar.
-template	Generate setup template files under ./templates/directory and exit.
-setup	Specifies the path to the setup file.
-stil_out_file	Specifies the STIL output file name.
-work_mode	[gen_pins—gen_timing] Specifies the mode under which to run pins generation or timing generation mode.

Chapter 5

VCDSTIL Tutorial

5.1 Introduction

5.1.1 VCDSTIL concept

The purpose of VCDSTIL is to convert event-based simulation results (which are design simulation files in VCD/EVCD format) to a cycle-based test in STIL format.

VCDSTIL divides the (E)VCD source file into cycles. Each different cycle is collected and translated to STIL waveform. The results include waveforms and vectors.

The purpose of this tutorial is to familiarize you with VCDSTIL. After completing this tutorial, you will be able to convert (E)VCD files to STIL files.

For this tutorial, a VCD file has been selected because the signal direction is not defined in a VCD file and needs to be defined manually. For Inout pins, I/O control signals (direction control) should be defined.

5.1.2 Modes of operation

VCDSTIL has three modes of operation, selected by the **work_mode** command line switch. By default, VCDSTIL generates STIL. However, in order to generate STIL, pins and timing data is needed. To prepare pins and timing files, you can run VCDSTIL in other work modes:

- **gen_pins** - generate a pins file containing list of pins and their directions. All pins are assumed to be input (in case of VCD source) or inout (EVCD source).
- **gen_timing** - generate a timing file, containing pin waveforms.
- default mode - generate STIL.

5.2 DUT Description - I8254

5.2.1 DUT overview

The Intel I8254 is a counter/timer device, it provides three independent 16-bit counters, each capable of handling clock inputs up to 10MHz.

For more information refer to the I8254 User Manual.

5.2.2 DUT pin definitions

The following is the list of the device signals direction and I/O control signals, as defined in the device specification:

Pin	Direction	I/O control
A0	Input	-
A1	Input	-
CLK0	Input	-
CLK1	Input	-
CLK2	Input	-
CS_	Input	-
D0	Inout	Controlled by D_dir
D1	Inout	Controlled by D_dir
D2	Inout	Controlled by D_dir
D3	Inout	Controlled by D_dir
D4	Inout	Controlled by D_dir
D5	Inout	Controlled by D_dir
D6	Inout	Controlled by D_dir
D7	Inout	Controlled by D_dir
GATE0	Input	-
GATE1	Input	-
GATE2	Input	-
OUT0	Output	-
OUT1	Output	-
OUT2	Output	-
RD_	Input	-
WR_	Input	-
D_dir	Input	-

Table 5.1: Device Direction Definitions

D_dir is an internal I/O control pin for D0-D7 signals Note:

When the D_dir I/O control signal value = 1, the Inout signal direction is Input.

When the D_dir I/O control signal value = 0, the Inout signal direction is Output.

5.2.3 DUT timing definitions

The device operates at 2.5MHz, which is why a tester cycle of 400ns has been selected. The rest of the definitions are generated automatically by VCDSTIL.

5.3 Using automated timing extraction

5.3.1 Creating setup file from template

VCDSTIL's behavior is controlled by the setup file. To see a complete list of setup file parameters, run VCDSTIL with the **template** command line switch:

```
>VCDSTIL -template
```

As a result, the **template** directory is created. This directory contains a template of the setup file **VCDSTIL_setup_template.py** and templates of data sheets (pins, timing, etc.).

5.3.2 Extracting pins information from VCD

The first step is preparing the pins file. This file contains a list of pins and their directions. As a starting point, let's use VCDSTIL to extract the pins information from the VCD file:

```
>VCDSTIL -setup setup_pins.py -work_mode gen_pins
```

The following is the setup file used for this run:

```
1 input_files = ['WR_RD_Mode0.vcd']
2 keep_design_hierarchy = 0
3 pins_file = 'pins.csv'
```

1. **Line 1:** `input_files = [ 'WR_RD_Mode0.vcd' ]` - The input VCD file.
2. **Line 2:** `keep_design_hierarchy = 0` - Remove module names from signal names. For example, the signal will be named CLK0 and not test1.U.CLK0.
3. **Line 3:** `pins_file = 'pins.csv'` - Pin definitions file (will be generated by VCDSTIL).

The log:

```
1 Processing Setup...
2 Done [setup]
3
4 Writing external files...
5   pins.csv
6 Done [writing]
```

Let's review the generated pins file. Initially, all pins are assumed to be input.

1	<VS Pins>	1			
2	Pin	Type	I/O Controls	State Map	SSP
3	D_dir	in			0
4	A0	in			0
5	A1	in			0
6	CLK0	in			0
7	CLK1	in			0
8	CLK2	in			0
9	CS_	in			0
10	D0	in			0
11	D1	in			0
12	D2	in			0
13	D3	in			0
14	D4	in			0
15	D5	in			0
16	D6	in			0
17	D7	in			0
18	GATE0	in			0
19	GATE1	in			0
20	GATE2	in			0
21	OUT0	in			0
22	OUT1	in			0
23	OUT2	in			0
24	RD_	in			0
25	WR_	in			0
26	</VS Pins>				

Figure 5.1: Pins File Generated by VCDSTIL

- **Pin** - Pin name.
- **Type** - Pin direction - in/out/inout.
- **I/O Control** - Direction control (for inout signals).
- **State map** - State map (advanced option, out of scope of this tutorial).
- **SSP** - Smart strobe positioning enable/disable (advanced option, out of scope of this tutorial).

The pins file should now be edited to assign correct pin directions and direction controls, as defined in DUT specifications:

1	<VS Pins>	1			
2	Pin	Type	I/O Controls	State Map	SSP
3	D_dir	in			0
4	A0	in			0
5	A1	in			0
6	CLK0	in			0
7	CLK1	in			0
8	CLK2	in			0
9	CS_	in			0
10	D0	inout	in:D_dir=1,out:D_dir=0		0
11	D1	inout	in:D_dir=1,out:D_dir=0		0
12	D2	inout	in:D_dir=1,out:D_dir=0		0
13	D3	inout	in:D_dir=1,out:D_dir=0		0
14	D4	inout	in:D_dir=1,out:D_dir=0		0
15	D5	inout	in:D_dir=1,out:D_dir=0		0
16	D6	inout	in:D_dir=1,out:D_dir=0		0
17	D7	inout	in:D_dir=1,out:D_dir=0		0
18	GATE0	in			0
19	GATE1	in			0
20	GATE2	in			0
21	OUT0	out			0
22	OUT1	out			0
23	OUT2	out			0
24	RD_	in			0
25	WR_	in			0
26	</VS Pins>				

Figure 5.2: Updated Pins File

5.3.3 Extracting timing from VCD

The next step is preparing the timing file, containing pin waveforms. To extract timing from the VCD file, VCDSTIL uses the pins file prepared in the previous step.

```
>VCDSTIL -setup setup_timing.py -work_mode gen_timing
```

```

1 1 input_files = ['WR_RD_Mode0.vcd']
2 2 keep_design_hierarchy = 0
3 3 pins_file = 'pins.csv'
4 4 timing_file = 'timing.csv'
5 5 cycle_width = '400ns'
6 6 start_time = '1600ns'
7 7 time_unit = 'ns'

```

- **Line 1:** `input_files = [ 'WR_RD_Mode0.vcd' ]` - The input VCD file.
- **Line 2:** `keep_design_hierarchy = 0` - Remove module names from signal names.
- **Line 3:** `pins_file = 'pins.csv'` - Pin definitions file.
- **Line 4:** `timing_file = 'timing.csv'` - Timing definitions file (will be generated by VCDSTIL).
- **Line 5:** `cycle_width = '400ns'` - Cycle period.
- **Line 6:** `start_time = '1600ns'` - Extraction start point; VCD events prior to 1600 ns will be ignored.
- **Line 7:** `time_unit = 'ns'` - Time unit to use in extracted timing.

The log:

```

1 Processing Setup...
2   pins.csv
3 Done [setup]
4
5 Analyzing Patterns...
6   WR_RD_Mode0.vcd
7 Done [patterns]
8
9 Running Timing Optimizer...
10 Done [optimizer]
11
12 Checking Test Limits...
13 Done [test limits]
14
15 Writing external files...
16   timing.csv
17 Done [writing]

```

The figure below contains a part of the generated timing file. For each pin, the file contains one or more waveforms, generated from VCD events.

1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	D_dir	U	0ns:U	waveform
4	D_dir	D	0ns:D	waveform
5	A0	U	0ns:U	waveform
6	A0	D	0ns:D	waveform
7	A1	U	0ns:U	waveform
8	A1	D	0ns:D	waveform
9	CLK0	D	0ns:D; 150ns:U	waveform
10	CLK1	U	0ns:U; 150ns:D	waveform
11	CLK2	D	0ns:D; 300ns:U	waveform
12	CLK2	U	300ns:U	waveform
13	CS_	D	0ns:D	waveform
14	CS_	U	0ns:U	waveform
15	D0	D	100ns:D	waveform
16	D0	U	100ns:U	waveform
17	D0	T	0ns:Z; 0ns:T; 200ns:L	waveform
18	D0	t	200ns:Z; 200ns:T	waveform
19	D0	H	200ns:Z; 200ns:H	waveform

Figure 5.3: Part of a Timing File Generated by VCDSTIL

- **Pin** - Pin name
- **WFC** - Waveform character to use in STIL timing (optional)
- **Waveform** - Waveform description
- **Condition** - Waveform matching condition. In automatically generated timing, the column contains the keyword **waveform**. It means that in STIL generation, the defined waveforms will be compared to VCD events, and the first to match VCD will be selected for use in output STIL pattern. VCDSTIL support other waveform matching methods, which will be described later.

5.3.4 Generate STIL

Now, you are ready to proceed to STIL generation, using pins and timing files. This run uses the same setup as in the previous step. However, this time the timing file, as well as the pins file, are used as inputs, and output is STIL workspace and pattern.

```
>VCDSTIL -setup setup_timing.py
```

The log:

```
1 Processing Setup...
2   pins.csv
3   timing.csv
4 Done [setup]
5
6 Analyzing Patterns...
7   WR_RD_Mode0.vcd
8 Done [patterns]
9
10 Running Timing Optimizer...
11 Done [optimizer]
12
13 Checking Test Limits...
14 Done [test limits]
15
16 Generating STIL...
17   Pattern:  ./stil/setup_timing/WR_RD_Mode0.pat.stil
18   Main File: ./stil/setup_timing.stil
19 Done [stil]
```

The following is an excerpt from the generated STIL file. Note that STIL signal directions are the same as those defined in the pins file, and the STIL timing matches waveforms defined in the timing file.

```

1  ...
2  Signals {
3  ...
4      "CLK0" In;
5      "CLK1" In;
6      "CLK2" In;
7      "CS_" In;
8      "D0" InOut;
9  ...
10 }
11 ...
12 Timing {
13     WaveformTable "WFT" {
14         Period '400ns';
15         Waveforms {
16             ...
17             "CLK0" {
18                 D { '0ns' D; '150ns' U; }
19             }
20             "CLK1" {
21                 U { '0ns' U; '150ns' D; }
22             }
23             "CLK2" {
24                 D { '0ns' D; '300ns' U; }
25                 U { '300ns' U; }
26             }
27             "CS_" {
28                 D { '0ns' D; }
29                 U { '0ns' U; }
30             }
31             "D0" {
32                 D { '100ns' D; }
33                 U { '100ns' U; }
34                 T { '0ns' Z; '0ns' T; '200ns' L; }
35                 t { '200ns' Z; '200ns' T; }
36                 H { '200ns' Z; '200ns' H; }
37             }
38             ...
39         }
40     }
41 }
42 ...
43 Pattern "WR_RD_Mode0" {
44     W "WFT";
45     V { "D_dir"=U; "A0"=U; "A1"=U; "CLK0"=D; "CLK1"=U; "CLK2"=D; "CS_"=D; "D0"=D;
46         "D1"=D; "D2"=D; "D3"=D; "D4"=U; "D5"=D; "D6"=D; "D7"=D; "GATE0"=U;
47         "GATE1"=U; "GATE2"=U; "OUT0"=L; "OUT1"=X; "OUT2"=X; "RD_"=U; "WR_"=D; }
48     V { "CLK2"=U; "WR_"=U; }
49     ...
50 }

```

5.4 Using user-defined waveforms

5.4.1 Why use user-defined waveforms

If VCDSTIL uses automatically generated timing, waveforms defined in the timing file are compared to VCD events of the current cycle; and the first pin waveform to match VCD appears in the output STIL pattern. Consequently, the resulting STIL matches exactly the source VCD. However, in some cases you know what timing you want to appear in STIL, and the timing you want is different from the automatically generated one. For example, in case of asynchronous VCD, automatic generation will lead to a large number of similar waveforms. You will probably want to replace these waveforms with a few user-defined waveforms.

The next section shows how to define a waveform differently from the automatically extracted VCD timing and how to force VCDSTIL to apply this waveform.

5.4.2 Modifying the extracted timing

Let's assume that pin **CLK0** should always drive low at 0 ns and drive high at 200ns. In addition, let's assume that pin **D0** should drive value at 150ns in input cycles and sample at 250 ns in output cycles. To force this behavior, the following waveforms need to be defined:

1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	D_dir	U	0ns:U	waveform
4	D_dir	D	0ns:D	waveform
5	A0	U	0ns:U	waveform
6	A0	D	0ns:D	waveform
7	A1	U	0ns:U	waveform
8	A1	D	0ns:D	waveform
9	CLK0	D	0ns:D; 200ns:U	true
10	CLK1	U	0ns:U; 150ns:D	waveform
11	CLK2	D	0ns:D; 300ns:U	waveform
12	CLK2	U	300ns:U	waveform
13	CS_	D	0ns:D	waveform
14	CS_	U	0ns:U	waveform
15	D0		150ns:F	true
16	D0		250ns:R	true

Figure 5.4: Timing File Showing User-defined Waveforms for Pin D0

For **CLK0**, only one waveform is defined (line 9). For this waveform, the **Condition** column contains **true**. This instructs VCDSTIL to use this waveform regardless of VCD timing. Such a waveform is further referred to as "forced".

In case of **D0**, two forced waveforms are defined, both with condition set to **true**. The first waveform (line 15) is for input cycles. The waveform instructs VCDSTIL

to take the value from VCD and drive it at 150ns. A special event, **F**, means every input state from VCD can be substituted in its place. Because this can result in the generation of more than one STIL waveform, you cannot define WFC; it will be generated automatically by VCDSTIL.

Since **F** only accepts input states, the waveform will not be applied to output cycles. For these cycles, another waveform is defined (line 16). This waveform uses the special event, **R**, which accepts only output states (use **S** to accept both input and output states).

Run VCDSTIL.

```
>VCDSTIL -setup setup_forced.py
```

The following is an excerpt from the generated STIL. Note that now the value is always driven/sampled where we wanted it to be.

```

1  ...
2      "CLKO" {
3          D { '0ns' D; '200ns' U; }
4      }
5  ...
6      "D0" {
7          D { '150ns' D; }
8          U { '150ns' U; }
9          L { '250ns' L; }
10         T { '250ns' T; }
11         H { '250ns' H; }
12     }
13  ...

```

5.5 Using conditional waveforms

5.5.1 Why use conditional waveforms

Aside from the separation between input and output cycles, forced waveforms (as used in the previous section) do have a mechanism controlling the selection of waveforms for a specific pattern cycle. When such a mechanism is needed, conditional waveforms should be used. Conditional waveforms differ from forced waveforms due to the fact that they have a condition, which consists of signal, value and time point. VCDSTIL checks value of the specified signal at the specified time point, and compares the result to the value you specified. It is possible to combine multiple checks, using AND/OR logical operations.

The next section shows how to define conditional waveforms.

5.5.2 Modifying the timing

Let's assume that pin **D1** should drive value at 150 ns in input cycles and sample at 250 ns in output cycles. However, when pin **CLK2** is high at the start of the cycle, driver should be switched off instead. To force this behavior, the following waveforms should be defined:

1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	D_dir	U	0ns:U	waveform
4	D_dir	D	0ns:D	waveform
5	A0	U	0ns:U	waveform
6	A0	D	0ns:D	waveform
7	A1	U	0ns:U	waveform
8	A1	D	0ns:D	waveform
9	CLK0	D	0ns:D; 200ns:U	true
10	CLK1	U	0ns:U; 150ns:D	waveform
11	CLK2	D	0ns:D; 300ns:U	waveform
12	CLK2	U	300ns:U	waveform
13	CS_	D	0ns:D	waveform
14	CS_	U	0ns:U	waveform
15	D0		150ns:F	true
16	D0		250ns:R	true
17	D1		150ns:F	CLK2=='D'@(t0)
18	D1		0ns:Z	CLK2=='U'@(t0)
19	D1		250ns:R;	true

Figure 5.5: Timing File Showing Conditional Waveforms for Pin D1

The waveform at line 17 will only be applied if **CLK2** is low at the start of the cycle (t0). Similarly, the waveform at line 18, which switches the driver off, will only be

applied if **CLK2** is high. To check the signal value at another point, for example, 5ns after cycle start, the desired offset can be specified (for example, change t0 to t0+5ns).

The following is an excerpt from the generated STIL:

```

1  ...
2      "D1" {
3          Z { '0ns' Z; }
4          D { '150ns' D; }
5          U { '150ns' U; }
6          H { '250ns' H; }
7          T { '250ns' T; }
8      }
9  ...
10 Pattern "WR_RD_Mode0" {
11     W "WFT";
12     V { "D_dir"=U; "A0"=U; "A1"=U; "CLK0"=D; "CLK1"=U; "CLK2"=D; "CS_"=D; "D0"=D;
13         "D1"=D; "D2"=D; "D3"=D; "D4"=U; "D5"=D; "D6"=D; "D7"=D; "GATE0"=U;
14         "GATE1"=U; "GATE2"=U; "OUT0"=L; "OUT1"=X; "OUT2"=X; "RD_"=U; "WR_"=D; }
15     V { "CLK2"=U; "D1"=Z; "WR_"=U; }
16     V { "CLK2"=D; "CS_"=U; "D1"=D; }
17     V { "CLK2"=U; "D1"=Z; }
18     ...
19 }
```

Chapter 6

Advanced Tutorials

Do these advanced tutorials only after you have completed the basic tutorial of Chapter 5

6.1 Using VCDSTIL with a spreadsheet (Excel/Open office)

In this tutorial we will use CSV files for editing the pins direction. The process and the results are the same as the first basic tutorial but here we define all the direction and timing of the pins in a CSV file.

What is a CSV File ?

The Comma Separated Value (CSV) file format is actually a text file format which can be edited as an Excel (Open Office) file.

Why use CSV (spreadsheet) Files ?

If you have a lot of pins in your design, it is sometimes easier to edit all of them in an Excel file.

6.1.1 Run VCDSTIL with CSV (spreadsheet) file

Change your directory to `./testcases/Extraction_Tutorial_Using_CSV`

Under this directory, you will find the following files:

- `imported_pins_after_editing.csv`
- `setup.py`
- `WR_RD_Mode0.vcd`

Step 1: Retrieve pin information from VCD and export it to a CSV file

First of all, we will export all pins from the VCD file into CSV file format. Then we will open the files and setup the direction of the pins.

For generating the CSV file from WR_RD_Mode0.vcd, run the following command:

```
> VCDSTIL -csv_export_pins imported_pins.csv WR_RD_Mode0.vcd
```

The following is an example of the generated log for this run:

```
1 Processing Setup...
2 [csv_export_pins] Exporting to 'imported_pins.csv'.
3 Done [setup]
```

Verify that the imported_pins.csv file was created.

Step 2: Update signals direction in a CSV file

Open the CSV file and update all signals direction according to the device specification.

In this tutorial we have done it for you. Just compare the two files to see the difference. See 'imported_pins.csv' (the generated CSV file) and the 'imported_pins_after_editing.csv' (after editing), for reference.

	A	B	C	D	E	F
1	VS Pin Attributes	1				
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform	StateMap
3	##HELP##	in/out/inout	in:pin=states,out:pin=states	time:state ... time:state <u>sp</u> =time	time:state ... time:state <u>sp</u> =time	map_name
4	D_dir	in				
5	A0	in				
6	A1	in				
7	CLK0	in				
8	CLK1	in				
9	CLK2	in				
10	CS_	in				
11	D0	in				
12	D1	in				
13	D2	in				
14	D3	in				
15	D4	in				
16	D5	in				
17	D6	in				
18	D7	in				
19	GATE0	in				
20	GATE1	in				
21	GATE2	in				
22	OUT0	in				
23	OUT1	in				
24	OUT2	in				
25	RD_	in				
26	WR_	in				

Figure 6.1: CSV file before editing

	A	B	C	D	E	F
1	VS Pin Attributes	1				
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform	StateMap
3	##HELP##	in/out/ <u>inout</u>	in:pin=states	out:pin=states	time:state ... time:state <u>sp</u> =time	time:state ... time:state <u>sp</u> =time
4	A0	in				
5	A1	in				
6	CLK0	in				
7	CLK1	in				
8	CLK2	in				
9	CS_	in				
10	D0	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
11	D1	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
12	D2	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
13	D3	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
14	D4	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
15	D5	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
16	D6	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
17	D7	<u>inout</u>	in:D <u>dir</u> =1,out:D <u>dir</u> =0			
18	GATE0	in				
19	GATE1	in				
20	GATE2	in				
21	OUT0	out				
22	OUT1	out				
23	OUT2	out				
24	RD_	in				
25	WR	in				

Figure 6.2: CSV File after Editing

We config the pins direction as defined in the device specification:

- D0-D7 signals will be inout and the I/O control signal for D0-D7 pins is the D_dir pin.
- Configure the OUT0-2 to be an output.
- Configure all other pins as an input.
- Remove the D_dir pin from the lists of pins because it is only a control pin for D0-D7.

Step 3: Run VCDSTIL setup file

Run the conversion using the following command:

```
> VCDSTIL -setup setup.py
```

The generated log for this run:

```
1 Processing Setup...
2 [csv_import_pins] Importing from 'imported_pins_after_editing.csv'.
3 Done [setup]
4
5 Analyzing Patterns...
6 WR_RD_Mode0.vcd
7 Done [patterns]
8
9 Running Timing Optimizer...
10 Done [optimizer]
11
12 Checking Test Limits...
13 Done [test limits]
14
15 Generating STIL...
16 Pattern: ./stil/setup/WR_RD_Mode0.pat.stil
17 Main File: ./stil/setup.stil
18 Done [stil]
19
```

Now, let us examine the contents of the configuration file, setup.py:

```
1
2 input_files = [ 'WR_RD_Mode0.vcd' ]
3
4 cycle_width = '400ns'
5
6 csv_import_pins( path= 'imported_pins_after_editing.csv')
7
```

6.1.2 Commands Explanation

In line 2: `input_files = [ 'WR_RD_Mode0.vcd' ]` - This is the input VCD file. **In line 4:** `cycle_width = '400ns'` - This is the STIL file period. **In line 6:** `csv_import_pins( path= 'imported_pins_after_editing.csv')` - Imports the CSV file which we edited, and replaces the `pin_attributes` command that we used in the first, basic tutorial.

6.2 How to use the Master file withste

This tutorial shows how to use a STIL master file with VCDSTIL.

You will find the tutorial under `./testcases/Master_Tutorial`, in which you will find the following files:

- `setup.py`
- `WR_RD_Mode0.vcd`
- `setup.stil`
- `master_file/Master_STIL.stil`

The `setup.stil` file is the waveform file, which was generated in the `Extraction_Basic_Tutorial` without the link to the pattern file in the end of the file.

In this tutorial, we want to reduce the number of edges of `OUT0..OUT2`.

In order to do that, do the following:

1. Edit `setup.stil` with the `ste` tool in order to mask `OUT0..OUT2`.
2. Run VCDSTIL when importing the Master STIL file.
3. Compare the results.

Step 1: Edit `setup.stil` with the `ste` tool in order to mask `OUT0..OUT2`

1. Open the file `setup.stil` with `ste` by writing

```
> ste setup.stil
```

2. Scroll down until you see the signals OUT0..OUT2.

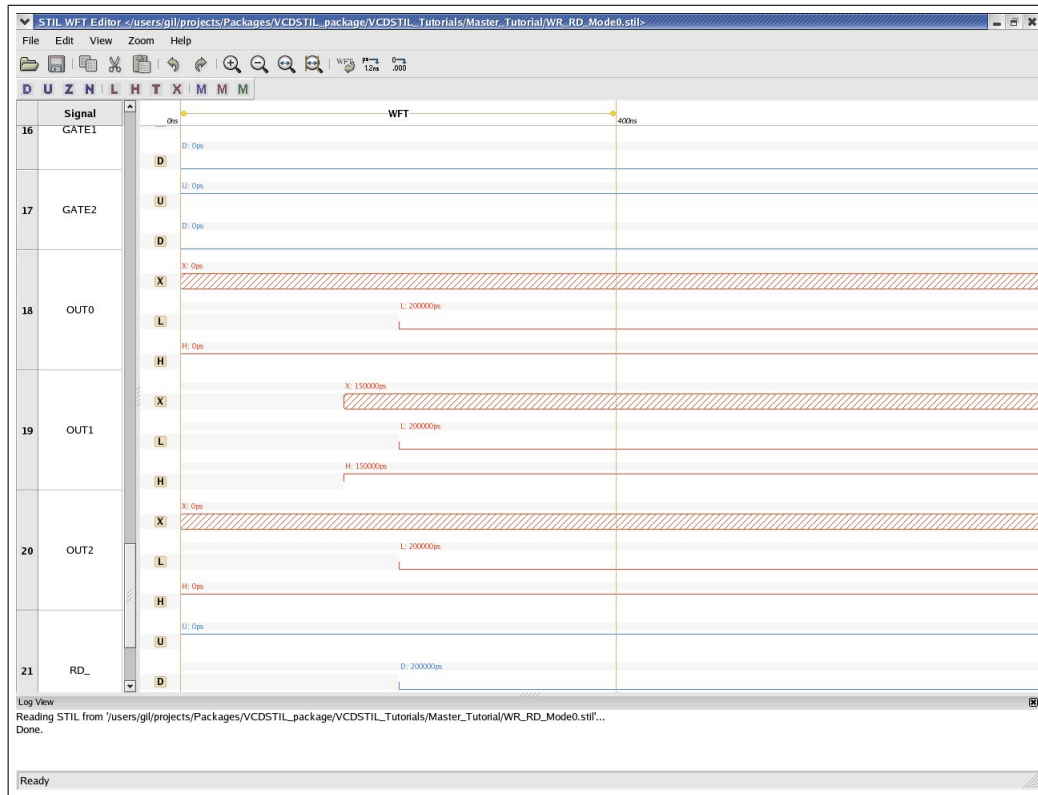


Figure 6.3: Signals OUT0..OUT2

3. Mark with a square the three signals, OUT0..OUT2, in order to mask all the events before the last event and then push the red button marked M (mask output).

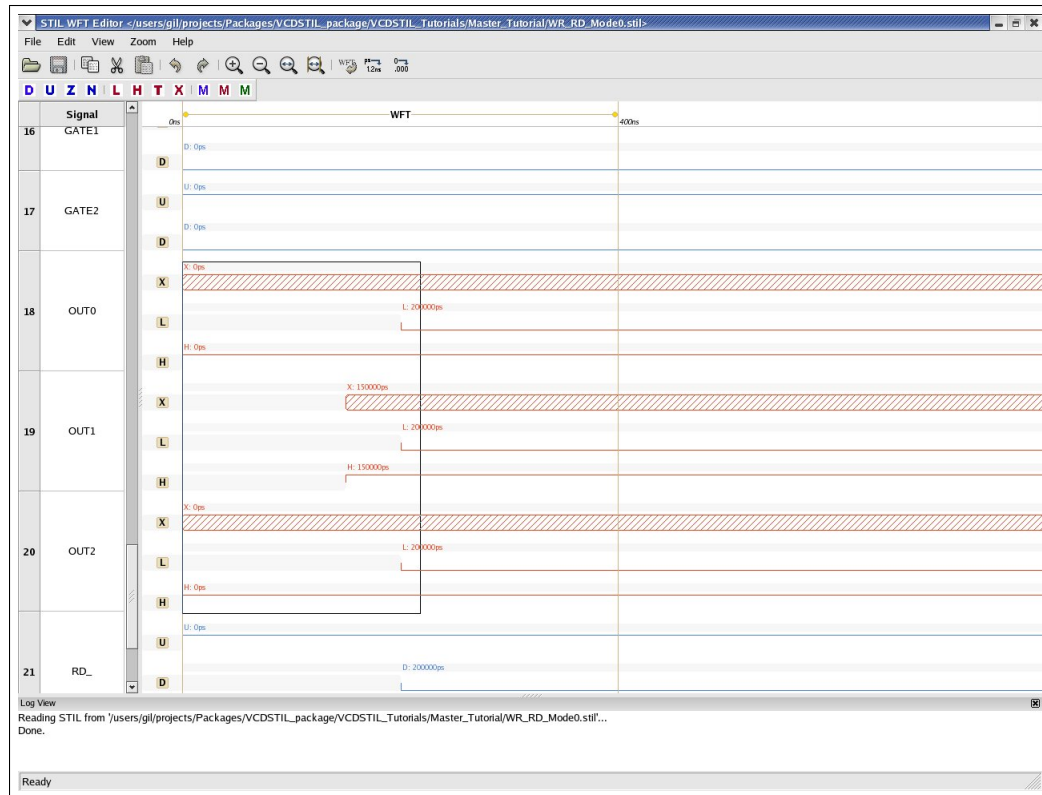


Figure 6.4: Masking the OUT0..OUT2 Signals

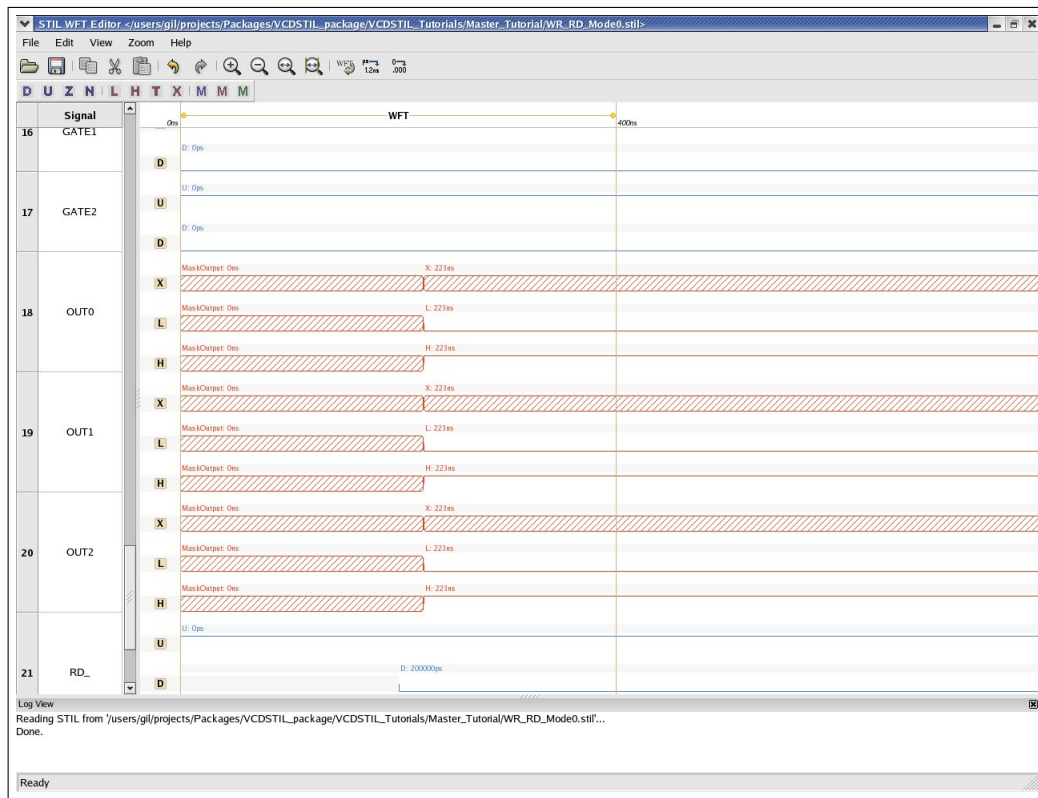


Figure 6.5: Masking the OUT0..OUT2 Signals

4. Select File -> Save As and save as Master_STIL.stil. Do not save the file as setup.stil. We will use this file for comparing later results.

You can compare the results or copy the master file in the master file directory and use it to run the setup file.

Step 2: Run VCDSTIL with Master file

Type the following command in order to run VCDSTIL:

```
>VCDSTIL -setup setup.py
```

See the generated log:

```

1 Processing Setup...
2 [stil_import] Reading STIL Master File './Master_STIL.stil'.
3
4 Done [setup]
5
6 Analyzing Patterns...
7 WR_RD_Mode0.vcd
8 Done [patterns]
9
10 Running Timing Optimizer...
11 Done [optimizer]
12
13 Checking Test Limits...
14 Done [test limits]
15
16 Generating STIL...
17 Pattern: ./stil/setup/WR_RD_Mode0.pat.stil
18 Main File: ./stil/setup.stil
19 Done [stil]
20

```

Now let's examine the contents of the configuration file, setup.py:

```

1 input_files = [ 'WR_RD_Mode0.vcd' ]
2
3 cycle_width = '400ns'
4
5 exclude_pins( pinlist = ['D_dir'] )
6
7 pin_attributes(
8     pinlist      = ['D?'],
9     type         = 'inout',
10    io_control    = 'in:D_dir=1,out:D_dir=0',
11 )
12
13 pin_attributes(
14     pinlist      = ['OUT?'],
15     type         = 'out',
16 )
17
18 pin_attributes(
19     pinlist      = ['A?','CS_','GATE?','RD_','WR_','CLK?'],
20     type         = 'in',
21 )
22
23 stil_import( path = './Master_STIL.stil' )

```

Explanation of Commands :

All the commands are identical to the basic tutorial, except:

In line 22: stil_import(path = './Master_STIL.stil') - import the Master STIL file which we edited. **Step 3: Compare the results**

Type the following command:

```

1 diff setup.stil ./stil/setup.stil

```

```

1 134,136c134,136
2 <          X { '0ps' Z; '0ps' X; }
3 <          L { '200000ps' Z; '200000ps' L; }
4 <          H { '0ps' Z; '0ps' H; }
5 ---
6 >          X { '223ns' X; }
7 >          L { '223ns' L; }
8 >          H { '223ns' H; }
9 139,141c139,141
10 <          X { '150000ps' Z; '150000ps' X; }
11 <          L { '200000ps' Z; '200000ps' L; }
12 <          H { '150000ps' Z; '150000ps' H; }
13 ---
14 >          X { '223ns' X; }
15 >          L { '223ns' L; }
16 >          H { '223ns' H; }
17 144,146c144,146
18 <          X { '0ps' Z; '0ps' X; }
19 <          L { '200000ps' Z; '200000ps' L; }
20 <          H { '0ps' Z; '0ps' H; }
21 ---
22 >          X { '223ns' X; }
23 >          L { '223ns' L; }
24 >          H { '223ns' H; }
25

```

You can see that the signals OUT0..OUT2 are masked, as we defined in the master file. You can see these differences clearly using the command *tkdiff*.

In general, the process of using master files gives you control over the number of waveforms. This is because the results contain fewer waveforms after we mask the unnecessary events.

6.3 Multi (E)VCD files with master file

This tutorial describes using a couple of (E)VCD files as input to VCDSTIL with the Master file.

It is recommend to do this tutorial after you have done the *How use Master file with ste* tutorial.

Change the directory to `./testcases/Multi_VCD_With_Master/`.

Under this directory are the following files:

- `setup.py`
- `WR_RD_Mode0.vcd`
- `WR_RD_Mode1.vcd`
- `WR_RD_Mode2.vcd`
- `Master_STIL.stil`

Run the conversion using the following command:

```
> VCDSTIL -setup setup.py
```

The generated log for this run:

```

1
2 Processing Setup...
3 [stil_import] Reading STIL Master File './Master_STIL.stil'.
4 [stil_import] Warning: STIL Master File './Master_STIL.stil':
5 Done [setup]
6
7 Analyzing Patterns...
8 WR_RD_Mode0.vcd
9 WR_RD_Mode1.vcd
10 WR_RD_Mode2.vcd
11 Done [patterns]
12
13 Running Timing Optimizer...
14 Done [optimizer]
15
16 Checking Test Limits...
17 Done [test limits]
18
19 Generating STIL...
20 Pattern: ./stil/setup/WR_RD_Mode0.pat.stil
21 Pattern: ./stil/setup/WR_RD_Mode1.pat.stil
22 Pattern: ./stil/setup/WR_RD_Mode2.pat.stil
23 Main File: ./stil/setup.stil
24 Done [stil]

```

As you can see, we have four files with one workspace and pattern file for each VCD file.

If we look at the results, we will see that all the pattern files are using the same timing (waveforms).

- `./stil/setup.stil` - STIL workspace file

- `./stil/setup/WR_RD_Mode0.pat.stil` - STIL pattern file for `WR_RD_Mode0.pat.stil`
- `./stil/setup/WR_RD_Mode1.pat.stil` - STIL pattern file for `WR_RD_Mode1.pat.stil`
- `./stil/setup/WR_RD_Mode2.pat.stil` - STIL pattern file for `WR_RD_Mode2.pat.stil`

Now let's examine the contents of the configuration file `setup.py`:

```
1 input_files = [ 'WR_RD_Mode0.vcd' , 'WR_RD_Mode1.vcd' , 'WR_RD_Mode2.vcd' ]
2
3 exclude_pins( pinlist = ['D_dir'] )
4
5 pin_attributes(
6     pinlist      = ['D?'],
7     type         = 'inout',
8     io_control   = 'in:D_dir=1,out:D_dir=0',
9 )
10
11 pin_attributes(
12     pinlist      = ['OUT?'],
13     type         = 'out',
14 )
15
16 pin_attributes(
17     pinlist      = ['A?','CS_','GATE?','RD_','WR_','CLK?'],
18     type         = 'in',
19 )
20
21 stil_import( path = './Master_STIL.stil' )
22
23 cycle_width = '400ns'
```

You can see we specified a couple of VCD files for input and we import the `Master_STIL.stil` file to `VCDSTIL`.

6.4 SSP tutorial

SSP (Smart Strobe Positioning) is a feature of VCDSTIL which automatically minimizes the number of waveforms and strobe positions created due to output waveform phases. The SSP feature only operates on output signals.

In this tutorial, you will familiarize yourself with the SSP feature in VCDSTIL.

Change the directory to `./testcases/SSP-Tutorial/`

Under this directory, you will find the following files:

- `setup.py`
- `WR_RD_Mode0.vcd`

At first we run the conversion without the SSP feature Run the following command:

```
>VCDSTIL -setup setup.py
```

See the generated log:

```
1 Processing Setup...
2 Done [setup]
3
4 Analyzing Patterns...
5   WR_RD_Mode0.vcd
6 Done [patterns]
7
8 Running Timing Optimizer...
9 Done [optimizer]
10
11 Checking Test Limits...
12 Done [test limits]
13
14 Generating STIL...
15   Pattern:  ./stil/setup/WR_RD_Mode0.pat.stil
16   Main File: ./stil/setup.stil
17 Done [stil]
```

Copy the main file which generated to `STIL_without_SSP.stil`.

Run the following command:

```
> cp ./stil/setup.stil STIL_without_SSP.stil
```

Open the `setup.py` file.

```

1 input_files = [ 'WR_RD_Mode0.vcd' ]
2
3 cycle_width = '400ns'
4
5 exclude_pins( pinlist = ['D_dir'] )
6
7 pin_attributes(
8     pinlist      = ['D?'],
9     type         = 'inout',
10    io_control    = 'in:D_dir=1,out:D_dir=0',
11    )
12
13 pin_attributes(
14     pinlist      = ['OUT?'],
15     type         = 'out',
16    )
17
18 pin_attributes(
19     pinlist      = ['A?','CS_','GATE?','RD_','WR_','CLK?'],
20     type         = 'in',
21    )
22
23 #ssp(pinlist = ['OUT?'])

```

As you can see, the last command of SSP is in comment format (`#`). Remove this comment and run it again using the following command:

```
>VCDSTIL -setup setup.py
```

Now all that is left to do is to compare the files. Run the following command:

```
>diff STIL_without_SSP.stil ./stil/setup.stil
```

```

1 135,136c135,136
2 <          L { '200000ps' Z; '200000ps' L; }
3 <          H { '0ps' Z; '0ps' H; }
4 ---
5 >          L { '300000ps' Z; '300000ps' L; }
6 >          H { '300000ps' Z; '300000ps' H; }
7 140,141c140,141
8 <          L { '200000ps' Z; '200000ps' L; }
9 <          H { '150000ps' Z; '150000ps' H; }
10 ---
11 >          L { '300000ps' Z; '300000ps' L; }
12 >          H { '300000ps' Z; '300000ps' H; }
13 145,146c145,146
14 <          L { '200000ps' Z; '200000ps' L; }
15 <          H { '0ps' Z; '0ps' H; }
16 ---
17 >          L { '300000ps' Z; '300000ps' L; }
18 >          H { '300000ps' Z; '300000ps' H; }
19

```

You can see that the SSP has chosen one sampling point for each OUT signal. You can see these differences clearly with the command *tkdiff*.



Figure 6.6: The Result STIL File without the SSP Feature



Figure 6.7: The Result STIL File with the SSP Feature

6.5 Input/Output Timing Definition Tutorial

In this tutorial you will learn how to manually configure the timing for pins in VCDSTIL. The object is that the STIL timing file will behave like this diagram:

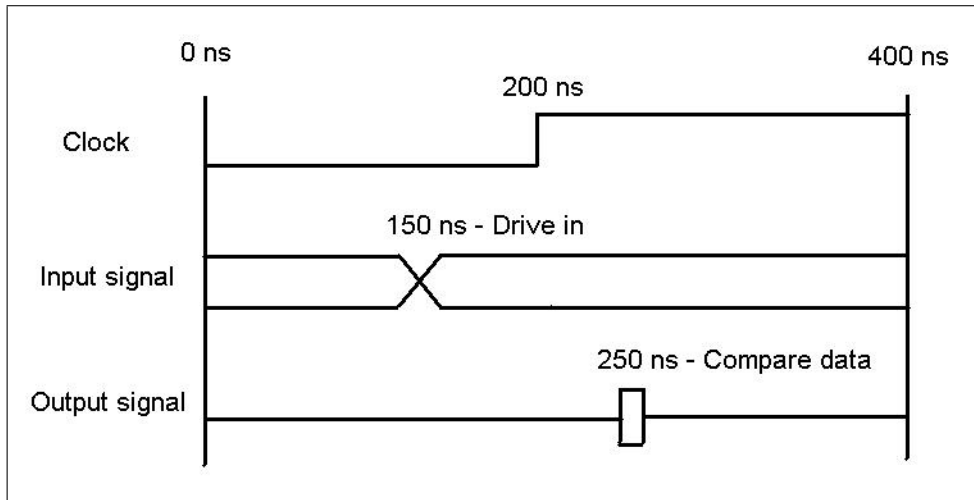


Figure 6.8: Diagram of Timing for Input/Output and Clock Signals

Explanation:

1. The device operates at 2.5MHz. As a result, the tester cycle is set to 400ns. Also, the clock is set to change at 200ns.
2. For input signals, we drive the sample data at 150ns, which is 50ns before the posedge of the clock.
3. For output signals we compare them at 250ns, which is 50ns after the posedge of the clock.

Change directory to `./testcases/Input_Output_Timing_Definition_Tutorial/`

Under this directory you will find the following files:

- `setup.py`
- `WR_RD_Mode0.vcd`

Look at the setup file :

```

1  input_files = [ 'WR_RD_Mode0.vcd' ]
2
3  cycle_width = '400ns'
4
5  exclude_pins( pinlist = ['D_dir'] )
6
7  pin_attributes(
8      pinlist      = ['CLK?'],
9      type         = 'in',
10     input_waveform = '0ns:D 200ns:U'
11 )
12
13 pin_attributes(
14     pinlist      = ['D?'],
15     type         = 'inout',
16     io_control   = 'in:D_dir=1,out:D_dir=0',
17     input_waveform = '150ns:S',
18     output_waveform = '250ns:S '
19 )
20
21 pin_attributes(
22     pinlist      = ['OUT?'],
23     type         = 'out',
24     output_waveform = '250ns:S'
25 )
26
27 pin_attributes(
28     pinlist      = ['A?','CS_','GATE?','RD_','WR_'],
29     type         = 'in',
30     input_waveform = '150ns:S'
31 )
32

```

As you can see, each pin is set to the timing which is defined in the timing specification.

- In line 10: for all the clock pins defined to drive low at 0ns and drive high at 200ns.
- In line 17-18: for all the inout pins defined for input state to drive the sample of the VCD at 150ns and for output state to compare the sample of the VCD at 250ns.
- In line 24: for all the output pins to compare the sample of the VCD at 250ns.
- In line 30: for all the input pins to drive the sample of the VCD at 150ns.

Run the following command:

```
>VCDSTIL -setup setup.py
```

See the generated log:

```

1 Processing Setup...
2 Done [setup]
3
4 Analyzing Patterns...
5   WR_RD_Mode0.vcd
6 Done [patterns]
7
8 Running Timing Optimizer...
9 Done [optimizer]
10
11 Checking Test Limits...
12 Done [test limits]
13
14 Generating STIL...
15   Pattern:  ./stil/setup/WR_RD_Mode0.pat.stil
16   Main File: ./stil/setup.stil
17 Done [stil]

```

Now look at the generated STIL file `./stil/setup.stil` and you will see the following content:

```

1 Timing {
2   WaveformTable "WFT" {
3     Period '400000ps';
4     Waveforms {
5       "A0" {
6         N { '150000ps' N; }
7         U { '150000ps' U; }
8         D { '150000ps' D; }
9       .....
10      .....
11       "D0" {
12         N { '150000ps' N; }
13         D { '150000ps' D; }
14         U { '150000ps' U; }
15         L { '250000ps' L; }
16         T { '250000ps' T; }
17         H { '250000ps' H; }
18       .....
19      .....
20       "OUT0" {
21         X { '250000ps' X; }
22         L { '250000ps' L; }
23         H { '250000ps' H; }
24       }
25       .....
26       .....

```

You can see that the STIL file timing is drive/compare at the points we have defined.

Here we have defined the timing definition for all the pins. You can also define timing for only a few of the pins.

Chapter 7

Advanced Topics

7.1 Controlling the Direction of Inout Signals

In VCD format, some states (0, 1, X, Z) are used to designate input and output events. Therefore, for bidirectional signals, a mechanism is needed to correctly translate VCD states to input or output STIL states. There are two ways to control the events direction:

- Using I/O control.
- Using a state map.

7.1.1 Controlling Direction Using I/O Control

In VCDSTIL translation, inout signals in VCD files can be determined by use of direction control in the pins file, as shown below. When 'D_dir' is low, 'D1' events will be translated to output events, and when 'D_dir' is high, to input events. It is possible to use two control signals, one for input and the other for output, or the same control signal for both directions.

1	<VS Pins>	1			
2	Pin	Type	I/O Controls	State Map	SSP
3	D_dir	in			0
4	D0	inout	in:D_dir=1,out:D_dir=0		0
5	</VS Pins>				

Figure 7.1: Pins File Showing the Use of IO Control

For detailed explanation about I/O controls, see [Direction Controls](#).

7.1.2 Controlling Direction Using Statemap

It is also possible to emulate I/O control by using a state map. The state map below explicitly defines events translation for signal 'D1'. It uses two arguments - 'D_dir' and the signal 'D1' itself (denoted by the '@' sign). Each line in the state map has two input states ('D_dir' and 'D1') and a single output state ('D1'). Events translation is defined in a way that makes 'D_dir' direction control signal for 'D1'. When 'D_dir' is low, 'D1' events will be translated to output events, when D_dir is high, to input events.

1	<VS Pins>	1.0			
2	Pin	Type	I/O Controls	State Map	SSP
3	D_dir	in			0
4				[test1.D_dir,@] 00->L 01->H 0Z->T 0X->X 10->D 11->U 1Z->Z 1X->N ..->X	
	.D1	inout			0
5	</VS Pins>				

Figure 7.2: Pins File Showing the Use of the State Map for Controlling Direction

7.2 Masking Output Signals

There is a number of ways to mask signal values:

- Using 'mask_transitions' command.
- Using the waveform condition.
- Using a state map.

7.2.1 Masking Transitions

One option is using the 'mask_transitions' transformation. The transformation file below instructs VCDSTIL to mask intervals of one cycle before and one cycle after each 'H'-to-'L' transition with 'X'.

1	<VS Transformations>	1.0	
2	Pin	Operation	Condition
3	mercedes_tb.clk	mask_transitions(H->L,1,1,X)	true
4	</VS Transformations>		

Figure 7.3: Transformations File Showing Transition Masking

7.2.2 Masking Using Conditions

You can prepare a special 'mask waveform' and define a condition for this waveform, which will instruct VCDSTIL to apply it when needed. For example, the timing file will cause VCDSTIL to use the 'X' waveform when the signal value is 'H', 300 ns into the cycle.

1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	OUT		0ns:X	OUT=='H'@(t0+300ns)
4	OUT		250ns:R;	true
5	</VS Timing>			

Figure 7.4: Timing File Showing Masking

For a detailed explanation of conditional waveforms, see [Conditional Waveforms](#).

7.2.3 Masking Using State Map

State maps control translation of (E)VCD events to STIL events. Therefore it can be used to mask certain events. In the examples below, a state map is used to keep events with compare high and compare low events, while converting all the rest to 'X'. Since VCD and EVCD formats have different states, different state maps are needed for different formats. The first example is for VCD conversion, the second for EVCD conversion. The arguments list in square brackets contains a single argument '@'. This means the 'OUT' signal itself is the only argument.

1	<VS Pins>	1.0			
2	Pin	Type	I/O Controls	State Map	SSP
3	OUT0	out		[@] 0->L 1->H .->X	0
4	</VS Pins>				

Figure 7.5: State Map for VCD Masking

1	<VS Pins>	1.0			
2	Pin	Type	I/O Controls	State Map	SSP
3	OUT0	out		[@] L->L H->H .->X	0
4	</VS Pins>				

Figure 7.6: State Map for EVCD Masking

7.3 Changing Signal Values

State maps are powerful tools for manipulating signal values. They enable the assignment of new values to signals, based on values of one or more signals. A simple case - masking of some values with 'X' - is shown above (see [Masking Using State Map](#)). Below is a more advanced example. The state map below changes the value of signal 'SIG3' to a logical OR between SIG1 and SIG2. The source is assumed to be EVCD. This time, the arguments list contains two signals ('SIG1', 'SIG2'), and accordingly each line of the state map has two input states. Note that the number of state map arguments is not limited, so the resulting value can be defined as a function of many arguments.

1	<VS Pins>	1.0			
2	Pin	Type	I/O Controls	State Map	SSP
3	SIG3	in		[SIG1,SIG2] DD->D DU->U UD->U UU->U ..->Z	0
4	</VS Pins>				

Figure 7.7: State Map Implementing Logical OR

7.4 Using Multiple Clock Domains

In order to implement multiple clock domains, use the ports file. Each line in the ports file defines a port, which is effectively a group with its own cycle width. The example below defines two ports. 'Port1' has a cycle of 200 ns and includes two signals 'A0' and 'A1'. 'Port2' has a cycle of 400ns and contains a single signal 'D0'.

1	<VS Ports>	1.0	
2	Port	Pins	Period
3	Port1	A0,A1	200ns
4	Port2	D0	400ns
5	</VS Ports>		

Figure 7.8: State Map Implementing Logical OR

The following is an excerpt from the generated STIL. For each port, its own STIL waveform table and pattern are generated.

```

1 Timing {
2     WaveformTable "WFT_Port1" {
3         Period '200ns';
4         Waveforms {
5             "A0" {
6                 U { '0.25*200ns' U; }
7                 D { '0.25*200ns' D; }
8             }
9             "A1" {
10                U { '0.25*200ns' U; }
11                D { '0.25*200ns' D; }
12            }
13        }
14    }
15    WaveformTable "WFT_Port2" {
16        Period '400ns';
17        Waveforms {
18            "D0" {
19                ...
20            }
21        }
22    }
23 }
24 ...
25 Pattern "vcd1_Port1" {
26     W "WFT_Port1";
27     V { "A0"=U; "A1"=U; }
28     ...
29 }
30
31 Pattern "vcd1_Port2" {
32     W "WFT_Port2";
33     V { "D0"=0; }
34     ...
35 }

```

For detailed explanation about ports, see [Multiport Mode](#).

7.5 Using Multiple Waveform Tables

By default, VCDSTIL generates STIL with a single waveform table. To generate multiple waveform tables, define them in the waveform tables file. For each waveform table, a name and a condition must be provided. For VCDSTIL to use a certain waveform table in a specific cycle, the condition for that waveform table must evaluate to true in that cycle.

In the following example, two waveform tables are defined. One will be used if signal CLK2 is low at cycle start, and the other if CLK is high.

1	<VS WFT Map>	1	
2	WFT	Port	Condition
3	MYWFT1		test1.U.CLK2=='D'@(t0)
4	MYWFT2		test1.U.CLK2=='U'@(t0)
5	</VS WFT Map>		

Figure 7.9: Defining Multiple Waveform Tables

Excerpt from the generated STIL:

```

1 Timing {
2     WaveformTable "MYWFT1" {
3         Period '400ns';
4         Waveforms {
5             "CLK2" {
6                 D { '0ps' D; '300000ps' U; }
7             }
8         }
9     }
10    WaveformTable "MYWFT2" {
11        Period '400ns';
12        Waveforms {
13            "CLK2" {
14                U { '300000ps' U; }
15            }
16        }
17    }
18 }
19 ...
20 Pattern "vcd1" {
21     W "MYWFT1";
22     V { "CLK2"=D; }
23     W "MYWFT2";
24     V { "CLK2"=U; }
25 ...
26 }

```

7.6 Optimizing Strobe Position

Smart Strobe Positioning (SSP) enables the minimization of the number of waveforms and strobe positions created due to output waveform phases. If SSP is enabled for a certain signal, VCDSTIL tries to locate a common strobe position for all output waveforms of that signal.

Consider, for example, waveforms { '10ns' H; '40ns' L; }, { '20ns' H; } and { '20ns' L; '50ns' H; }, with a period of 60ns. VCDSTIL tries to locate the largest interval where:

- all waveforms have no events,
- all waveforms are in valid output state.

In this case, the interval is 20 ns to 40 ns. Then, a compare event is positioned in the middle of the interval. Therefore, the process results in the generation of waveforms { '30ns' H; } and { '30ns' L; }.

Below is an example demonstrating the use of SSP.

```

1  $timescale 1ns $end
2  $scope module tb $end
3  $var port 1 <11 data $end
4  $upscope $end
5  $enddefinitions $end
6  #0
7  $dumpvars
8  pX 5 0 <11
9  $end
10 #105
11 pH 0 6 <11
12 #125
13 pL 6 0 <11
14 #149
15 pH 0 6 <11
16 #174
17 pL 6 0 <11
18 #185
19 pH 0 6 <11
20 #205
21 pL 6 0 <11
22 $vcdclose 220 $end

```

Automatic translation of the source (start time 100 ns, cycle width 20 ns) results in the following STIL timing, with a large number of output waveforms:

```

1  Timing {
2    WaveformTable "WFT" {
3      Period '20ns';
4      Waveforms {
5        "tb.data" {
6          H { '5ns' Z; '5ns' H; }
7          L { '5ns' Z; '5ns' L; }
8          h { '9ns' Z; '9ns' H; }
9          l { '14ns' Z; '14ns' L; }
10         }
11       }
12     }
13   }

```

To enable SSP, the value in the SSP column of the pins file must be changed from 0 to 1. Then, generate timing (i.e., run VCDSTIL with `-work_mode gen_timing`) to generate a new timing file, and proceed to STIL generation. This time, the following timing will be generated:

```
1 Timing {
2     WaveformTable "WFT" {
3         Period '20ns';
4         Waveforms {
5             "tb.data" {
6                 0 { '0ns' Z; '14ns' H; }
7                 1 { '0ns' Z; '14ns' L; }
8             }
9         }
10    }
11 }
```
